

CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA IN INFORMATICA

**Implementazione di modelli
TinyML su ESP32 per il rilevamento
di anomalie da dati vibrazionali con
infrastruttura backend centralizzata**

RELATORI:

Prof. Michele Pasqua

LAUREANDO:

Secchi Pietro Giampaolo

ANNO ACCADEMICO 2025 - 2026

Indice

Introduction	1
1 Analisi del Contesto e Requisiti	2
2 Architettura Generale del Sistema	3
2.1 Panoramica dell'Architettura di Sistema	3
2.2 Scelta del Protocollo di Rete	4
2.2.1 HTTP	4
2.2.2 MQTT	4
2.3 Scelta del Protocollo di Presentazione Dati	5
2.3.1 Protocolli basati sul testo	5
2.3.2 Protocolli basati su Rappresentazione Binaria	6
2.4 Architettura backend	7
2.5 Architettura Edge/Firmware	7
3 Sviluppo del Livello Edge	9
3.1 Hardware	9
3.1.1 Consumo Energetico	11
3.1.2 Sintesi dei Consumo di Sistema	14
3.2 Software	14
3.2.1 Struttura principale	14
3.2.2 4-way Buffer	16
3.3 Inferencer	18
3.3.1 Architettura deil' <i>ensemble</i>	20
3.3.2 Gestione dei sottomodelli	20
3.4 Ottimizzazioni	21

3.4.1	Ottimizzazione della conversione in float	21
4	Modelli utilizzati	25
4.0.1	Modello Memoria	25
5	Sviluppo del Backend e Integrazione	29
6	Testing, Deployment e Risultati	31
7	Conclusioni e Sviluppi Futuri	33
	Bibliografia	35

Introduzione

Capitolo 1

Analisi del Contesto e Requisiti

Capitolo 2

Architettura Generale del Sistema

2.1 Panoramica dell'Architettura di Sistema

Il sistema è stato sviluppato con una architettura distribuita di tipo *edge-to-cloud*, progettata per disaccoppiare le fasi di acquisizione dati vibrazionali, l'inferenza di modelli *on edge* per il rilevamento delle anomalie e l'addestramento di questi modelli. Il sistema è formato da quattro livelli principali:

- **edge**: dispositivo formato da un microcontrollore e un sensore MEMS a 6 assi, acquisisce i dati vibrazionali, li emette sporadicamente al cloud ed esegue l'inferenza di modelli per valutare se i dati sono anomali
- **rete**: i dati vengono trasmessi *wireless* usando il protocollo MQTT
- **backend**: allena i modelli basandosi sui dati ricevuti dai sensori
- **storage**: abilita la persistenza dei dati storici

2.2 Scelta del Protocollo di Rete

Sono stati valutati due protocolli per la trasmissione dei dati, questi sono **HTTP** e **MQTT**.

2.2.1 HTTP

- **Vantaggi:**

- Ideale per l'integrazione con servizi tradizionali client-server.
- Permette trasferimento di file di grandi dimensioni attraverso meccanismi di *chunking*.
- Ha una latenza più bassa rispetto a MQTT

- **Svantaggi:**

- **Overhead elevato:** gli *header* di ciascun pacchetto introduce un consumo maggiore di risorse rispetto a MQTT [Adriansyah et al., 2026].
- Poco adatto a sistemi embedded o dispositivi con risorse limitate di memoria e calcolo.

2.2.2 MQTT

- **Vantaggi:**

- **Efficienza:** l'architettura *publish/subscribe* e l'overhead ridotto (header fino a 2 byte) lo rendono ottimale per invii di dati sporadici.
- **Affidabilità:** la presenza di tre livelli di Quality of Service (QoS) garantisce la corretta consegna dei messaggi anche con connessioni instabili.
- Basso impatto computazionale e consumo energetico ridotto.

- **Svantaggi:**

- **Gestione limitata di file pesanti:** non è concepito per il trasferimento efficiente di payload di grandi dimensioni.

- **Necessario Broker:** necessita di un broker che gestisce la comunicazione tra gli endpoint
- **Latenza maggiore:** l'intermediazione del broker e i meccanismi di QoS introducono ritardi rispetto a comunicazioni dirette.

Per la leggerezza del protocollo e per il sistema a *publish/subscribe* è stato scelto MQTT, tuttavia la parte più critica è la trasmissione dei modelli, essendo di dimensione non trascurabile, per questo tipo di comunicazione si sarebbe potuta usare una architettura ibrida dove via MQTT l'edge device viene notificato del caricamento dei nuovi modelli che verrebbero poi scaricati via una richiesta GET di HTTP usando le funzioni di *chunking* del payload.

2.3 Scelta del Protocollo di Presentazione Dati

È necessaria una Lingua Franca per la rappresentazione dei dati condivisa che permette ai vari dispositivi nella rete di codificare e decodificare i dati in un formato condiviso, come descrive il livello 6 del modello *ISO/OSI* [ISO/IEC 7498-1:1994, 1994]. Le scelte si dividono in protocolli basati su testo (XML [Bray et al., 2008] /JSON [Bray, 2017]) e protocolli a rappresentazione binaria (CBOR [Bormann and Hoffman, 2020]/BSON [MongoDB, Inc., 2009]/Protocol Buffers [Varda, 2008]/MessagePack [Furuhashi, 2013]/Avro [Apache Software Foundation, 2011])

2.3.1 Protocolli basati sul testo

XML nasce come linguaggio di marcatura per documenti complessi basato su tag ed elementi ricchi di metadati Il JSON è un formato minimale per lo scambio di dati basato su coppie chiave-valore e array. Il JSON risulta notevolmente più leggero (30-40% di dimensione in meno [McNally,]), leggibile e rapido da elaborare via codice rispetto alla prolissità e alla complessità strutturale dell'XML.

2.3.2 Protocolli basati su Rappresentazione Binaria

Di natura sono più leggeri rispetto ai protocolli basati sul testo, evitando la conversione del testo in dati usabili dall'applicativo, quindi sono preferibili in sistemi limitati che giovano ad avere un risparmio di tempo di calcolo e di memoria.

Tabella 2.1: Confronto formati di serializzazione binaria

Caratteristica	CBOR	BSON	Protobuf	MessagePack	Avro
Origine / Focus	IETF / IoT e risorse minime	MongoDB / Storage e traversal DB	Google / RPC: velocità e peso	Furuhashi / Alternativa JSON	Apache / Schema evolution
Schema	Opzionale	Opzionale	Obbligatorio (.proto)	Opzionale	Obbligatorio (JSON/IDL)
Posizione Schema	Assente o est. (CDDL)	Assente	File .proto separato	Assente	Nei file / Registry
Auto-descrittivo	Sì	Sì	No	Sì	No (serve schema)
Codifica base	JSON + Tag	JSON esteso	Tag / ID numerici campo	JSON + tipi ext	Ordine campi schema
Estensibilità	Tag IANA	Tipi custom DB	Estensioni/opzioni proto	Tipi ext	Regole evoluzione
Evoluz. Schema	Implicita (salto tag)	Implicita	Esplicita (stabilità ID)	Implicita (salto ext)	Esplicita (alias/default)
Dimensione	Compatta	Variabile (anche grande)	Molto compatta	Compatta	Molto compatta
Velocità	Alta	Alta (traversal DB)	Massima (RPC)	Alta	Alta
Standard	IETF RFC 8949	De facto (MongoDB)	De facto (Google)	Community spec	Apache Project
Usi Principali	IoT, CoAP, sicurezza	MongoDB	RPC, Microservizi	Caching, RPC, stream	Big Data (Kafka, Hadoop)

Fonte: [McNally,].

Data la tabella la migliore scelta è *CBOR* (Concise Binary Object Representation), per i seguenti motivi: è un protocollo *royalty-free* e *open source*, è stato sviluppato dalla base per essere usato in contesti IoT per usare poca memoria e permette di criptare i dati nativamente. Viene usata come libreria sul microcontrollore *espressif/cbor* [Espressif Systems, c],

derivata da *tinycbor* [Intel Corporation and Macieira, 2015], ed è impiegato anche *zcbor* [Øyvind Rønningstad and Nordic Semiconductor, 2021], un programma che permette di tradurre lo schema CDDL direttamente in codice C ottimizzato per la codifica e/o decodifica dei pacchetti in struct.

2.4 Architettura backend

il backend è formato da servizi ospitati su container Docker, il che permette di rendere il servizio modulare e facilmente riproducibile. I servizi docker sono:

- **EMQX:** [EMQ Technologies Inc., 2024] Un broker MQTT, versione Community, selezionato perchè include funzionalità *event driven*, come il salvataggio automatico in un database sql quando riceve un pacchetto.
- **Controller:** un servizio scritto in python che gestisce il *lifecycle* dei sensori, incluso l'allenamento dei modelli.
- **Postgresql;** [PostgreSQL Global Development Group, 2024] un database relazionale, scelto per la familiarità pregressa con tale tecnologia, che ha permesso di ridurre i tempi di sviluppo e concentrarsi direttamente sugli obiettivi del lavoro."

2.5 Architettura Edge/Firmware

Per garantire reattività nell'acquisizione, l'inferenza e l'invio dei dati, il firmware è strutturato su un modello concorrente che sfrutta l'architettura dual-core del microcontrollore, ovvero un ESP32-S3, scelto per le alte prestazioni.

Come sensore MEMS viene usato il modulo *STEVAL-MKI245KA* [?], di STMicroelectronics, un kit di valutazione del circuito integrato *ISM330BX* [?], concesso in uso per le attività di ricerca da Iotinga SRL.

Il sistema operativo usato è *FreeRTOS* [?], un sistema operativo runtime minimale che espone le funzionalità base di un sistema operativo, come la creazione di task, la gestione della rete e del *file system*.

Come *framework* di sviluppo viene usato ESP-IDF scelto per il controllo più a basso livello, rispetto ad Arduino Core, delle periferiche hardware e la gestione nativa dei task FreeRTOS.

Capitolo 3

Sviluppo del Livello Edge

3.1 Hardware

Vengono usati:

- **ESP32-S3** con modulo PSRAM da 16MB e memoria flash da 32MB
- **STEVAL-MKI245KA** in modalità di comunicazione SPI
- **Adafruit 4682** lettore sd in modalità di comunicazione SPI

3.1.1 Consumo Energetico

ESP32-S3 (Microcontrollore)

Tabella 3.1: ESP32-S3 Power Consumption – CPU Active, 25 °C

CPU Frequency	Peripherals	Data Access	Consumption
80 MHz	×	32 bit	33.1 mA
80 MHz	✓	32 bit	47.3 mA
80 MHz	×	128 bit	41.8 mA
80 MHz	✓	128 bit	56.3 mA
240 MHz	×	32 bit	66.2 mA
240 MHz	✓	32 bit	81.3 mA
240 MHz	×	128 bit	91.7 mA
240 MHz	✓	128 bit	107.9 mA

Tabella 3.2: ESP32-S3 Low-Power Modes, 25 °C

Mode	Condition	Consumption
Light-sleep	CPU e digital peripherals in clock-gating, Wi-Fi disattivato	0.240 mA
Deep-sleep	RTC memory e RTC peripherals alimentati	0.008 mA

Tabella 3.3: ESP32-S3 Wi-Fi Power Consumption, 25 °C

Mode	Configuration	Consumption
Transmitter	802.11b, 1 Mbps, $P_{OUT} = +21$ dBm	340 mA
Transmitter	802.11g, 54 Mbps, $P_{OUT} = +19$ dBm	291 mA
Transmitter	802.11n, HT20, MCS7, $P_{OUT} = +18.5$ dBm	283 mA
Transmitter	802.11n, HT40, MCS7, $P_{OUT} = +18$ dBm	286 mA
Receiver	802.11b/g/n, HT20	88 mA
Receiver	802.11n, HT40	91 mA

Il profilo energetico usato é *Modem-sleep mode*, rilasciato da Espressif [Espressif Systems, 2024b], dove il modem viene attivato solo quando necessario, quindi il consumo effettivo é:

- in **Modem TX**: CPU Active/All RTC/Wi-Fi, durante la comunicazione dei dati (286 mA)
- in **Modem RX**: CPU Active/All RTC/Wi-Fi, durante la ricezione di modelli, comandi e configurazioni dei dati (91 mA)

- in **Modem sleep**: CPU Active/All RTC/No Wi-Fi, durante la fase di sola inferenza, scrittura su sd o lettura del sensore (81.3 mA)

Il sistema acquisisce campioni da 12 byte (16bit per valore, 3 per accelerometro e 3 per giroscopio) ad una frequenza di 3.84 kHz (46.08 kB/s), trasmettendo un blocco ogni 4096 campioni. La dimensione di ciascun blocco è quindi

$$N_{\text{TX}} = 4096 \cdot 12 = 49152 \text{ byte.} \quad (3.1)$$

Il periodo tra due trasmissioni in modalità *continua*, ovvero vengono caricati tutti i *datapoint*

$$T_{\text{block}} = \frac{4096}{3840} = 1.067 \text{ s.} \quad (3.2)$$

Assumendo la velocità teorica massima di 150 Mbit/s per 802.11n HT40 MCS7, il tempo necessario per un blocco è

$$T_{\text{TX}} = \frac{49152 \cdot 8}{150 \cdot 10^6} = 2.62 \text{ ms.} \quad (3.3)$$

Assumendo una corrente di 286 mA la potenza per singola trasmissione è

$$E_{\text{TX}} = \frac{3.3 \cdot 0.286 \cdot 2.62 \times 10^{-3}}{3.6} \approx \boxed{6.87 \times 10^{-4} \text{ mWh}}. \quad (3.4)$$

Il duty cycle della trasmissione è

$$D_{\text{TX}} = \frac{2.62 \text{ ms}}{1.067 \text{ s}} \approx 0.246\%. \quad (3.5)$$

La corrente media associata al Wi-Fi è quindi

$$I_{\text{WiFi,avg}} = 286 \text{ mA} \cdot 0.00246 \approx 0.70 \text{ mA.} \quad (3.6)$$

Considerando inoltre 81.3 mA per CPU e periferiche attive, la corrente media complessiva risulta

$$I_{\text{avg}} = 81.3 + 0.70 \approx 82.0 \text{ mA.} \quad (3.7)$$

I valori ottenuti rappresentano una stima teorica basata sulla velocità massima di 150 Mbit/s e non tengono conto degli overhead del protocollo Wi-Fi e dei livelli superiori.

ISM330BX

Modalità Operativa	Dettagli Funzionalità	Canali e	Corrente Tipica (I_{DD})
Combo High-Performance	Accelerometro + Giroscopio attivi ad alto ODR		$\approx 0.60 \text{ mA}$ (600 μA)
Power-Down	Dispositivo spento, interfacce seriali in attesa		3 μA – 5 μA

Tabella 3.4: Consumi elettrici ISM330BX [STMicroelectronics, 2024].

il sensore viene usato in modalità *High-Performance* a 3.84kHz, con coda FIFO, ovvero il sensore salva in una regione di memoria le letture, il che permette di leggere a blocchi i dati dal sensore.

Adafruit 4682

Il breakout Adafruit 4682 è una scheda passiva, quindi il consumo è dato unicamente dalla scelta della scheda microSD. Consumo scheda SD:

Current	V_{DD}	Max.
Peak	3.6 V	300 mA
Average	3.6 V	250 mA

Tabella 3.5: consumi di una scheda microSD, microSDXC, Class 10, U1, 64GB, modello SDCIT/64GB, marca Kingston. [Kingston Technology,]z.

Analogamente al caso precedente, si riutilizzano gli stessi risultati per il consumo del controllore. Considerando un data rate di 46.08 kB/s, e una velocità minima di scrittura di 10 MB/s per una SD Classe 10, il duty cycle della scrittura o lettura è pari a circa 0.46%.

Assumendo un consumo di 250 mA durante la scrittura o lettura il consumo medio della SD è quindi circa 1.15 mA. Sommando il consumo della CPU e delle periferiche, pari a 81.3 mA, si ottiene

$$I_{\text{avg,SD}} \approx 82.45 \text{ mA}, \quad P_{\text{avg,SD}} \approx \boxed{272.1 \text{ mW}}. \quad (3.8)$$

3.1.2 Sintesi dei Consumo di Sistema

Sommando i consumi si ha un consumo medio teorico di $\approx 83.75 \text{ mA}$, o $\approx 276.4 \text{ mW}$, e un consumo di picco di $\approx 618.9 \text{ mA}$ o $\approx 2042 \text{ mW}$. Il che significa che una batteria 18650 da circa 9Wh, riuscirebbe ad alimentare il device per 32 ore

Tecniche di Ottimizzazione energetica

32 ore sono la stima per la modalità con il massimo consumo, ovvero la lettura, salvataggio, invio dei dati e l'inferenza basata su suddetti dati. Una volta che la *backend* riceve abbastanza dati per allenare modelli validi si può mettere il device in varie modalità risparmio energetico configurabile

Modalità	CPU	Duty Cycle	Corrente	Potenza	Autonomia
Active	240 MHz	100%	83.75 mA	276.4 mW	32.6 h
Active	80 MHz	100%	49.75 mA	164.2 mW	54.8 h
Light-sleep	240 MHz	3.33%	3.70 mA	12.22 mW	30.7 giorni
Light-sleep	80 MHz	3.33%	2.57 mA	8.48 mW	44.2 giorni
Deep-sleep	240 MHz	3.33%	3.48 mA	11.47 mW	32.7 giorni
Deep-sleep	80 MHz	3.33%	2.34 mA	7.74 mW	48.5 giorni

Tabella 3.6: Consumi e autonomia stimati nelle modalità, batteria da 9 Wh. Il duty cycle è del 3.33% (2 minuti di attività ogni ora, ovvero circa 112 segmenti da 4096 datapoint). Il duty cycle del Wi-Fi viene scalato a 0.41%, il lettore SD è spento durante le fasi di sleep.

3.2 Software

3.2.1 Struttura principale

Il codice del firmware é scritto in C/C++ usando il framework ESP-IDF. Il sistema giova dell'architettura dual core del microcontrollore ESP32-S3 [Espressif Systems, 2024b] Ci sono 6 task/thread principali

Nome Task	Priorità	Core Assegnato
harvester	5	Core 1
inferencer	4	Core 1
uploader	3	Core 0
dump_reader	3	Core 0
sd_writer	2	Core 0
mqtt_task	5 (Default)	Core 0 (Default)

Tabella 3.7: Priorità delle Task FreeRTOS e selezione Core

Tutte le task nel core 0 hanno grandi ritardi di I/O perché richiedono chiamate di rete oppure la lettura e scrittura di file su scheda SD. Mentre l’inferencer nel Core 1 è cpu-bound per la maggior parte del tempo e IO-bound se deve caricare i modelli dalla rete oppure dalla scheda SD. L’harvester viene messo nello stesso Core dell’inferencer perché vogliamo dare precedenza alla lettura dei dati in maniera prioritaria.

harvester (Core 1, Priorità 5) Esegue l’interrogazione della coda FIFO del sensore ISM330BX via SPI per estrarre i campioni di accelerometro e giroscopio. La priorità è massima per permettere alla coda FIFO di non riempirsi mai e non andare in *overflow* [STMicroelectronics, 2024], il che significherebbe perdere dei *datapoints* contigui.

inferencer (Core 1, Priorità 4) Calcola il magnitudo e carica dello spettrogramma STFT 2D. Esegue i modelli del *ensemble* (insieme di modelli) e calcola il valore delle anomalie.

uploader (Core 0, Priorità 3) Gestisce la serializzazione in formato CBOR e la trasmissione dei dati. Divide l’array di campioni float in pacchetti più piccoli per rientrare nei limiti dei buffer di rete dell’MQTT. E invia i dati al broker MQTT.

dump_reader (Core 0, Priorità 3) Viene istanziato temporaneamente in caso di trigger di replay, ovvero se la backend richiede un upload di tutti i dati storici salvati. Legge i file binari dalla scheda SD e li reinserisce nella pipeline di elaborazione, nello stesso modo in cui farebbe *harvester*, che viene messo in pausa nel caso venga richiesto il replay. È un processo IO-bound, quindi messo nel Core 0

sd_writer (Core 0, Priorità 2) Prende i dati, sia i datapoint e i risultati dell'inferenza) e li scrive sulla scheda SD in formato binario.

mqtt_task (Core 0, Priorità 5 – Default di Sistema) Gestito in background dalla libreria client di MQTT di Espressif (*espressif/mqtt* [Espressif Systems, a]).

3.2.2 4-way Buffer

La struttura dati principale che unisce le varie task é un *4-way buffer*, una variante di un *Ping Pong buffer* con 4 buffer al posto di 2. Tutte le task leggono e scrivono dagli slot del quad buffer, usando dei semafori per la sincronizzazione, e per evitare controlli attivi.

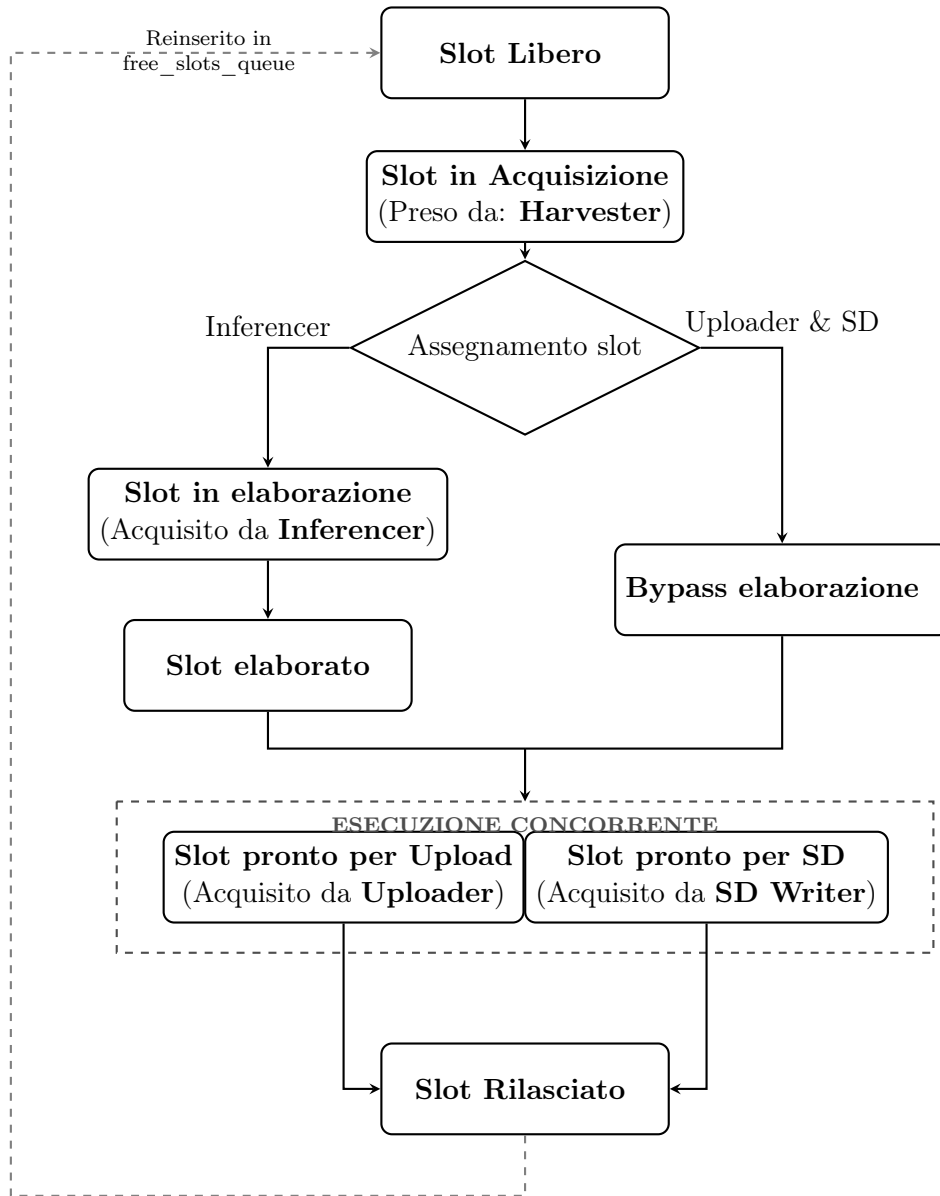


Figura 3.2: Grafo del ciclo di vita di uno slot in modalità

L'harvester prende il primo slot che trova quindi, dopo essere stato popolato dai dati, viene acquisito dall'inferencer, e poi `sd_writer` e `uploader` salvano su `sd` e caricano sul cloud il segmento con i dati inferenziali, oppure se l'inferencer sta elaborando un altro slot, viene assegnato direttamente al `uploader` e al `sd_writer`, che salvano e caricano lo slot senza dati inferenziali. La ragione per cui abbiamo bisogno di 4 buffer è che questo garantisce che 2 siano sempre occupati dal harvester che salva i dati come un classico ping pong buffer senza interruzioni, uno sia occupato

dall'inferencer e un altro dai uploader e sd_writer, in questo modo nessun task deve aspettarne un altro.

Il sistema é predisposto anche per altre modalità. Le modalità di upload di dati, configurabili dalla backend runtime sono:

- *Continuous*: ovvero quello nel grafico sopra, il device carica sul cloud tutti i dati che riesce, elaborati o no
- *AnomalyOnly*: il device carica sul cloud il segmento solo se risulta in un anomalia
- **AnomalyOrPeriodic**: il device carica sul cloud il segmento solo se risulta in un anomalia, ma carica con una certa cadenza anche dati non anomali per permettere un allenamento continuo dei modelli
- **ContinuousScoreRawAnomaly**: il device carica sempre il risultato dell'inferenza, ma carica anche il segmento solo se si tratta di un anomalia

Il device espone alla backend dei comandi di dump opzionali, questi sono *dump*, che carica sul cloud tutti i dati dalla scheda SD senza farne l'inferenza (harvester e inferencer disabilitati) e *dump_i* che carica tutti i dati, ma calcolandone anche le anomalie (harvester disabilitato).

Nella configurazione del device é possibile anche disabilitare del tutto la scheda sd per risparmiare energia se non si ha bisogno di chiamare il comando *dump* dei dati.

3.3 Inferencer

L'inferencer é la task che gestisce la vita dei modelli e la loro inferenza, appoggiandosi ad una struttura dati chiamata *ensemble* che aiuta la gestione dei modelli e astrae la gestione della gestione della libreria tflite micro [Espressif Systems, b]. Ai dati viene applicata una pipeline che riduce la dimensionalità, ovvero una riduzione euclidea di magnitudo per accelerazione e giroscopio:

$$\text{Accel Magnitude}[n] = \sqrt{\text{accel}_x[n]^2 + \text{accel}_y[n]^2 + \text{accel}_z[n]^2}$$

$$\text{Gyro Magnitude}[n] = \sqrt{\text{gyro}_x[n]^2 + \text{gyro}_y[n]^2 + \text{gyro}_z[n]^2}$$

questo permette di mantenere solo i dati vibrazionali, indipendentemente dall'orientamento di montaggio del sensore.

Successivamente viene applicata ad entrambi una funzione a finestra di hanning (overlap 75%) e una trasformazione di Fourier. Viene scelta la finestra di Hanning perchè offre un buon compromesso tra risoluzione e contenimento dello spectral leakage [Harris, 1978].

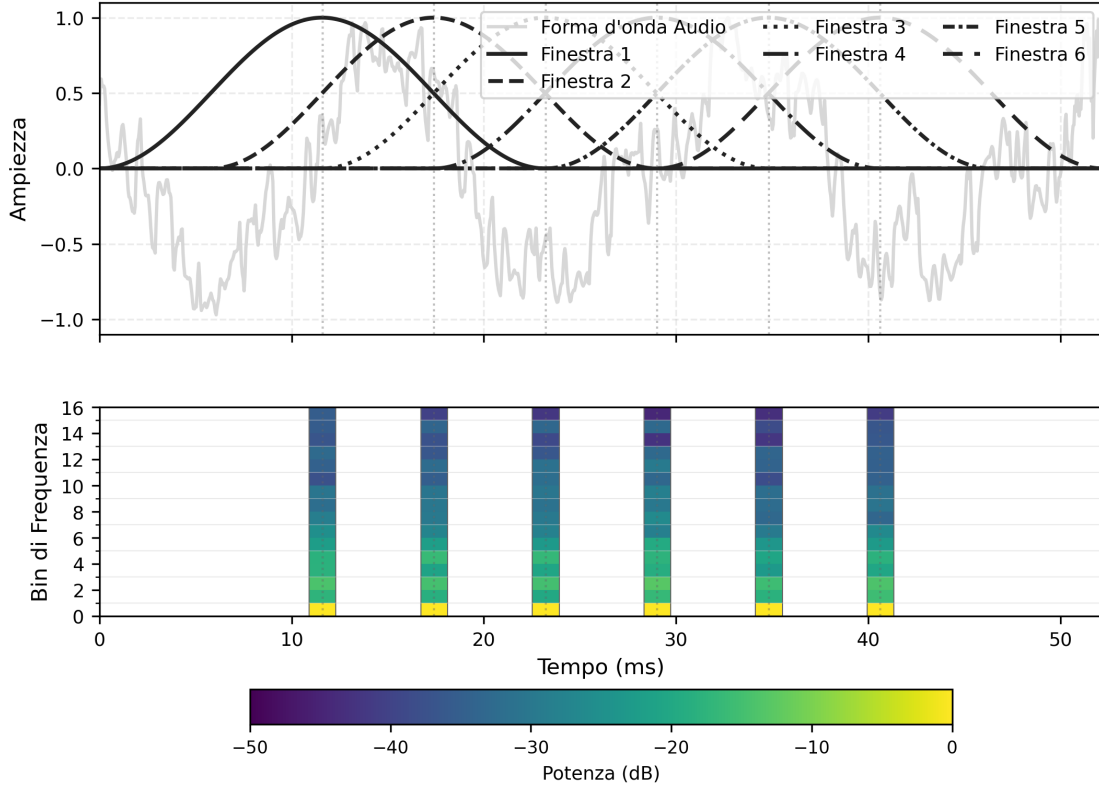


Figura 3.3: Applicazione finestra di hanning e trasformata di fourier

Infine gli stft del giroscopio e accelerazione vengono concatenati verticalmente, così da creare un solo segmento di dati.

3.3.1 Architettura dell'*ensemble*

L'architettura è formata da 3 tipi di modelli disposti in maniera simil-MIXAE [Zhang et al., 2017]:

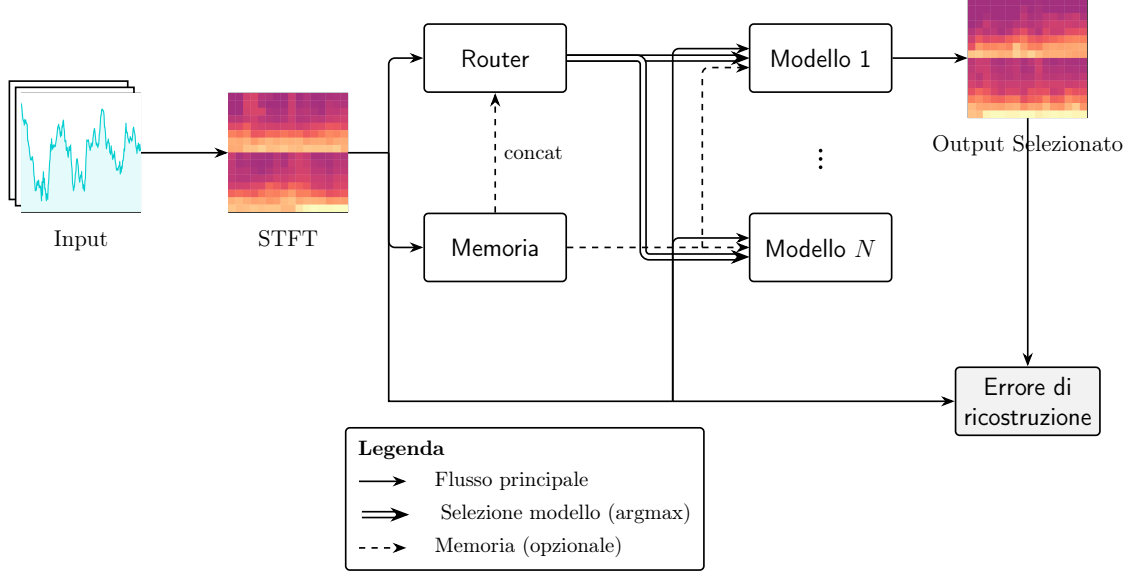


Figura 3.4: Architettura del *ensemble*.

- 1 **Router**: prende in input un segmento e lo stato della memoria e decide quale autoencoder dovrà processarlo
- 1 **Memoria**: memoria, opzionale per modello. Utile per modelli non convoluzionali, è un modo per aggiungere informazioni sul passato.
- N **autoencoder**: ci sono N (max 16), autoencoder che prendono il segmento e provano a ricostruirlo, l'errore di ricostruzione indica il livello di anomalia, configurabile tra MSE e MAE, può essere usato qualsiasi modelli rappresentabile con grafi nativi tflite micro

3.3.2 Gestione dei sottomodelli

In memoria non possono alloggiare tutti i sottomodelli, se si scelgono architetture troppo complesse, quindi è necessario avere un modo di caricare a bisogno i sottomodelli. Per questo l'inferencer dispone di una struttura dati di cache di modelli, creando così una gerarchia di recupero dati. Prima viene interrogata la

cache alla ricerca del modello, poi la scheda sd se abilitata, poi il cloud attraverso una richiesta MQTT. Una volta caricato un nuovo modello, se la memoria non è sufficiente, viene decrementata di uno la dimensione della cache dei modelli, liberando così una parte di memoria.

3.4 Ottimizzazioni

. Vengono usate varie funzioni ottimizzate per l'ESP32S3

- **dsp_opt_int16_to_float**, custom, permette di convertire un buffer da int16 a float con l'istruzione `FLOAT.S` di Xtensa LX7 [xte, 2019]
- **dsps_fft2r_fc32 (Radix-2 Complex FFT)**: calcola la fft sui dati del sensore. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_fft2r_init_fc32 (Inizializzazione FFT)**: Precalcola le tabelle dei fattori rotazionali (*twiddle factors*) per evitare runtime calcoli di seni e coseni, molto pesanti. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_bit_rev_fc32 (Bit Reversal)**: Esegue il riordinamento dell'array in output di `dsps_fft2r_fc32`. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_wind_hann_fc32 (Generazione Finestra di Hanning)**: Calcola e applica la finestra di Hann. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_mulc_fc32 (Moltiplicazione Vettore-Scalare)**: Esegue la moltiplicazione di un array float per un numero. Fornita da ESP-DSP [Espressif Systems, 2024a].

3.4.1 Ottimizzazione della conversione in float

Il sensore restituisce dati in tipo int16, ma le funzioni di calcolo si aspettano dati in formato float da -1 a 1, quindi dobbiamo convertirli usando la funzione `dsp_opt_int16_to_float` che sfrutta l'istruzione assembly `FLOAT.S`

L'istruzione `FLOAT.S`

L'approccio *naive* sarebbe fare un ciclo for facendo un cast di ogni valore del buffer in float e poi moltiplicarlo per $1/32768.0$:

$$x_{\text{float}} = (\text{float})x_{\text{int16}} \cdot \frac{1}{32768.0} \quad (3.9)$$

Questo approccio fa eseguire nell'FPU del controllore due istruzioni: un cast hardware (`FLOAT.S` con fattore di scala nullo) ed una moltiplicazione in floating point (`MUL.S`).

Per ottimizzare si è sfruttato l'ISA del coprocessore matematico dell'architettura Xtensa LX7 dell'ESP32-S3. Come scritto nel manuale dell'ISA [xte, 2019], `FLOAT.S` possiede un campo per fare lo shift del risultato:

$$\text{FR}[r] \leftarrow \text{floats}(\text{AR}[s]) \cdot 2^{-t} \quad \text{con } t \in [0, 15] \quad (3.10)$$

Impostando $t = 15$, la FPU procede col casting int16-float e la divisione per $2^{15} = 32768$ nello stesso ciclo di clock. Si elimina così l'istruzione di moltiplicazione e l'overhead di memoria per la costante.

Ottimizzazione della pipeline

Come definito nei modelli di schedulazione di GCC per l'architettura Xtensa [Free Software Foundation, 2024], le istruzioni di caricamento sulla memoria `load` e di conversione (`fconv`) hanno latenze:

- **Caricamento** (`xtensa_memory`): 2 cicli di clock.
- **Conversione** (`xtensa_fconv`): 2 cicli di clock (fully-pipelined, con throughput di 1 istruzione per ciclo).

Quindi se viene eseguita la conversione con `FLOAT.S` elemento per elemento, ovvero con fattore di unrolling $U=1$, la cpu dovrà continuare ad aspettare che la FPU scriva i risultati nei registri prima di poter salvare il valore in memoria. Per evitare la latenza di pipeline abbiamo bisogno di un fattore di unrolling U maggiore uguale della latenza di pipeline [Hennessy and Patterson, 2017].

La latenza che abbiamo è di 4 cicli (2 di caricamento+2 di conversione), Quindi scegliamo $U=4$.

Lo pseudocodice dell’algoritmo ottimizzato con inline assembly è riportato nel Codice 3.1.

Listing 3.1: Algoritmo di conversione ottimizzato a 4 vie con assembly Xtensa

```
void dsp_opt_int16_to_float(const int16_t *src, float *dest,
    int len) {
    int i = 0;
    for (; i + 3 < len; i += 4) {
        // genera istruzioni L16SI,
        // carica e converte in 2 cicli
        int32_t s0 = src[i+0]; int32_t s1 = src[i+1];
        int32_t s2 = src[i+2]; int32_t s3 = src[i+3];
        float f0, f1, f2, f3;

        asm volatile (
            "float.s_ %[f0], _ %[s0], _ 15_ \n\t"
            "float.s_ %[f1], _ %[s1], _ 15_ \n\t"
            "float.s_ %[f2], _ %[s2], _ 15_ \n\t"
            "float.s_ %[f3], _ %[s3], _ 15_"
            : [f0] "=f" (f0), [f1] "=f" (f1), [f2] "=f" (f2),
              [f3] "=f" (f3)
            : [s0] "r" (s0), [s1] "r" (s1), [s2] "r" (s2),
              [s3] "r" (s3)
        );

        dest[i+0] = f0; dest[i+1] = f1;
        dest[i+2] = f2; dest[i+3] = f3;
    }
}
```

Validazione sperimentale benchmark

Per validare l’analisi è stato condotto un benchmark sull’ESP32-S3 misurando i cicli di clock effettivi tramite il registro CCOUNT. Ogni benchmark è stato fatto su 20000 volte un buffer da 4096 valori, per simulare il vero utilizzo.

Configurazione	Cicli/op	Tempo buffer	Ottimizzazione	Speedup
Assembly x1	17.08	291.50 μ s	29.24%	1.41×
Assembly x2	10.57	180.39 μ s	56.21%	2.28×
Assembly x4	8.26	140.97 μs	65.78%	2.92×
Assembly x8	9.01	153.77 μ s	62.68%	2.68×
Naive C x1	24.14	411.99 μ s	0.00%	1.00×
Naive C x2	17.58	300.03 μ s	27.17%	1.37×
Naive C x4	14.32	244.39 μ s	40.68%	1.68×
Naive C x8	12.82	218.79 μ s	46.90%	1.88×
Naive C x16	11.95	203.95 μ s	50.50%	2.02×
Naive C x32	22.55	384.85 μ s	6.59%	1.07×
Naive C x64	23.34	398.29 μ s	3.33%	1.03×

Tabella 3.8: Risultati del su ESP32-S3 a 240 MHz

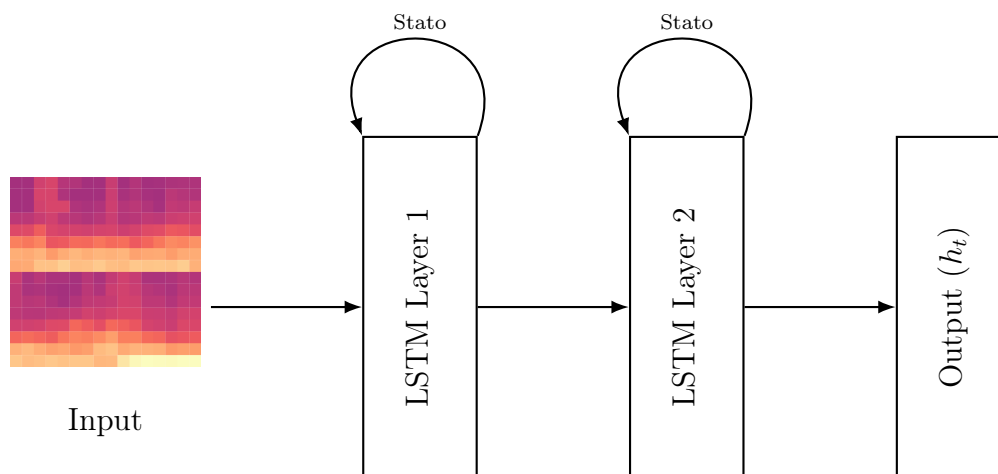
Si può notare che il metodo più veloce è Assembly con $U=4$, il che conferma l'analisi teorica. Si può anche notare che il metodo naive più veloce è con $U=16$, questo perchè la pipeline di calcolo naive è lunga 9 cicli perchè viene scomposta in 4 istruzioni, ovvero L16SI (2 cicli), FLOAT.S (2 cicli, senza shift), MUL.S (4 cicli), SSI (Store Single Precision, 1 ciclo), per un totale di 9 cicli totali. U deve essere quindi maggiore o uguale a 9, tuttavia per allineamento alla dimensione del bus, usiamo la potenza di due maggiore più vicina, ovvero 16.

Capitolo 4

Modelli utilizzati

Come abbiamo detto ci sono 3 tipi di modelli, il modello memoria, il modello router, e N sottomodelli autoencoder.

4.0.1 Modello Memoria



LSTM Memory Backbone utilizza dei layer a Long Short Term Memory (LSTM) [Hochreiter and Schmidhuber, 1997], che permettono agli altri sottomodelli ricordare dati passati. Riceve in input i singoli segmenti dello spettrogramma e fornisce in output una rappresentazione latente dei dati. Il modello memoria è condiviso con tutti i modelli che lo usano, quindi gli altri modelli devono essere allenati dopo il modello memoria.

Modello router

Il Router classifica i dati per selezionare l'autoencoder idoneo. Le architetture usate sono:

1. **Fully connected:** Rete neurale densa (Multilayer Perceptron, MLP)
2. **STFT MCNN:** Multi-branch Convolutional Neural Network. Applica reti convoluzionali parallele per estrarre caratteristiche dei dati diverse, usando kernel di dimensioni diverse [Zhang et al., 2016].
3. **1D CNN** Rete convoluzionale (multilivello) ad un kernel per livello che scansiona lo spettrogramma, ha un MLP finale per la classificazione. [Kiranyaz et al., 2015]

Modelli autoencoder

Provano a ricostruire il segnale di input, la ragione per cui sono utili nel contesto dell'anomaly detection è che riescono a ricostruire il segnale di input solo se sono stati allenati con dati i a quelli che dovranno poi ricostruire, questo fa sì che se incontrano dati mai visti non riusciranno a ricostruire in maniera affidabile l'input, e quindi possiamo assumere che quell'input è diverso in struttura dai dati in cui è stato allenato.

1. **Autoencoder denso:** Usa semplici layer densi, essendo di fatto un MLP con dei layer di bottleneck, che formano lo spazio latente, tra l'encoder e il decoder
2. **Variational autoencoder:** Un VAE è, a livello strutturale, un autoencoder denso, tuttavia nella fase di training, si forza lo spazio latente ad essere formato da una distribuzione normale $\mathcal{N}(0, 1)$ di $\mathcal{N}$ dimensioni. Questo permette allo spazio latente di non avere buchi, ovvero zone che producono in fase di decode dati incoerenti [Kingma and Welling, 2013a].
3. **Convolutional Autoencoder 1D:** Usa layer convoluzionali in fase di encode, e layer deconvoluzionali in fase di decode. Utilizza come input lo spettrogramma interpretato come una serie multivariata, dove i singoli bin di frequenza fungono da canali di input. Il filtro convoluzionale scorre esclusivamente lungo l'asse orizzontale.

4. **Convolutional Autoencoder 2D:**

Usa layer convoluzionali in fase di encode, e layer deconvoluzionali in fase di decode. Utilizza come input lo spettrogramma interpretato come una vera e propria immagine spaziale a singolo canale.

5. **Convolutional LSTM Autoencoder:** Combina layer convoluzionali (per le feature spaziali) e celle LSTM (per le feature temporali) per catturare anomalie spazio-temporali [Shi et al., 2015].

Capitolo 5

Sviluppo del Backend e Integrazione

Capitolo 6

Testing, Deployment e Risultati

Capitolo 7

Conclusioni e Sviluppi Futuri

Bibliografia

- [xte, 2019] (2019). *Xtensa Instruction Set Architecture (ISA) Reference Manual*. Cadence Design Systems, Inc., rev. 2.0 edition. Chapter 8 (Floating-Point Coprocessor Option), p. 431.
- [Adafruit Industries, 2020] Adafruit Industries (2020). *Adafruit Micro SD SPI or SDIO Card Breakout Board - 3V ONLY! (Product ID 4682)*. Adafruit Industries. Scheda breakout passiva a 3.3V priva di LDO e level shifter.
- [Adriansyah et al., 2026] Adriansyah, S., Sumarno, S., Nurfuha, U., Giffari, A., Khoirunisa, T., and Subarkah, W. (2026). Performance analysis of mqtt and http protocols on low-power esp32 devices for iot applications. *CoreID Journal*, 4:38–45.
- [Advanced Monolithic Systems, 2012] Advanced Monolithic Systems (2012). *AMS1117: 1A Low Dropout Voltage Regulator Datasheet*. Advanced Monolithic Systems. Riferimento per la corrente di quiescenza (Iq) tipica dei regolatori LDO generici.
- [Apache Software Foundation, 2011] Apache Software Foundation (2011). Apache Avro specification. <https://avro.apache.org/docs/current/specification/>.
- [Bormann and Hoffman, 2020] Bormann, C. and Hoffman, P. E. (2020). Concise Binary Object Representation (CBOR). RFC 8949, RFC Editor.
- [Bray, 2017] Bray, T. (2017). The JavaScript Object Notation (JSON) data interchange format. RFC 8259, RFC Editor.
- [Bray et al., 2008] Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., and Yergeau, F. (2008). Extensible Markup Language (XML) 1.0 (fifth edition). W3C recommendation, W3C.
- [EMQ Technologies Inc., 2024] EMQ Technologies Inc. (2024). EMQX: The

- open-source distributed MQTT broker. <https://www.emqx.io/>. Accessed: 2026-08-25.
- [Espressif Systems, a] Espressif Systems. Esp-mqtt api reference - esp32. ESP-IDF Programming Guide.
- [Espressif Systems, b] Espressif Systems. esp-tflite-micro. <https://components.espressif.com/components/espressif/esp-tflite-micro>.
- [Espressif Systems, c] Espressif Systems. espressif/cbor: Cbor library for esp-idf. <https://components.espressif.com/components/espressif/cbor>. Componente per ESP-IDF basato su TinyCBOR.
- [Espressif Systems, 2024a] Espressif Systems (2024a). Esp-dsp: Espressif dsp library. <https://github.com/espressif/esp-dsp>. Repository ufficiale GitHub.
- [Espressif Systems, 2024b] Espressif Systems (2024b). *ESP32-S3 Series Datasheet*. Espressif Systems. Documentazione ufficiale per consumi e caratteristiche radio/sleep.
- [Free Software Foundation, 2024] Free Software Foundation (2024). Gnu compiler collection (GCC) target description for Xtensa (xtensa.md). <https://github.com/gcc-mirror/gcc/blob/master/gcc/config/xtensa/xtensa.md>. Accessed: August 2026.
- [Furuhashi, 2013] Furuhashi, S. (2013). MessagePack: It's like JSON, but fast and small. <https://msgpack.org/>.
- [Harris, 1978] Harris, F. J. (1978). On the use of windows for harmonic analysis with the discrete fourier transform. *Proceedings of the IEEE*, 66(1):51–83.
- [Hennessy and Patterson, 2017] Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6th edition.
- [Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- [Intel Corporation and Macieira, 2015] Intel Corporation and Macieira, T. (2015). Tinycbor: Concise binary object representation (cbor) library. <https://github.com/intel/tinycbor>.
- [ISO/IEC 7498-1:1994, 1994] ISO/IEC 7498-1:1994 (1994). Information Technology – Open Systems Interconnection – Basic Reference Model: The Basic Model.

- Standard ISO/IEC 7498-1:1994, International Organization for Standardization, Geneva, Switzerland.
- [Kingma and Welling, 2013a] Kingma, D. P. and Welling, M. (2013a). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
- [Kingma and Welling, 2013b] Kingma, D. P. and Welling, M. (2013b). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
- [Kingston Technology,] Kingston Technology. *Industrial microSD Card*. Kingston Technology. Accessed: 2026-08-25.
- [Kiranyaz et al., 2015] Kiranyaz, S., Ince, T., and Gabbouj, M. (2015). Real-time patient-specific ecg classification by 1-d convolutional neural networks. *IEEE Transactions on Biomedical Engineering*, 63(3):664–675.
- [Masci et al., 2011] Masci, J., Meier, U., Cireşan, D., and Schmidhuber, J. (2011). Stacked convolutional auto-encoders for hierarchical feature extraction. In *International Conference on Artificial Neural Networks*, pages 52–59. Springer.
- [Mattia et al., 2021] Mattia, F. D., Galeone, P., Simoni, M. D., and Ghelfi, E. (2021). A survey on gans for anomaly detection.
- [McNally,] McNally, W. *The CBOR, dCBOR, and Gordian Envelope Book*.
- [Melichov, 2024] Melichov, M. (2024). Anomaly-detection. <https://github.com/maxmelichov/Anomaly-Detection>.
- [MongoDB, Inc., 2009] MongoDB, Inc. (2009). BSON (binary JSON) serialization format. <https://bsonspec.org/>.
- [Ng et al., 2011] Ng, A. et al. (2011). Sparse autoencoder. Technical report, Stanford University.
- [PostgreSQL Global Development Group, 2024] PostgreSQL Global Development Group (2024). *PostgreSQL: The World’s Most Advanced Open Source Relational Database*. PostgreSQL Global Development Group. Accessed: 2026-08-25.
- [Rifai et al., 2011] Rifai, S., Vincent, P., Muller, X., Glorot, X., and Bengio, Y. (2011). Contractive auto-encoders: Explicit invariance during feature extraction. In *Proceedings of the 28th International Conference on Machine Learning*, pages 833–840.
- [Saha et al., 2022] Saha, S. S., Sandha, S. S., and Srivastava, M. (2022). Machine learning for microcontroller-class hardware: A review. *IEEE Sensors Journal*, 22(22):21362–21390.

- [SD Association, 2022] SD Association (2022). *SD Specifications Part 1 Physical Layer Simplified Specification*. SD Association. Version 9.00.
- [Shi et al., 2015] Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-K., and Woo, W.-c. (2015). Convolutional lstm network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems*, pages 802–810.
- [STMicroelectronics, 2024] STMicroelectronics (2024). *ISM330BX: 6-axis IMU with High-Bandwidth Accelerometer, Sensor Fusion and AI for Industrial Applications*. STMicroelectronics. Datasheet di specifica elettronica e registri operativi.
- [Turkish Technology Team Foundation (T3),] Turkish Technology Team Foundation (T3). Power consumption (esp32-s3).
- [Varda, 2008] Varda, K. (2008). Protocol Buffers: Google’s data interchange format. <https://protobuf.dev/>.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008.
- [Vincent et al., 2008] Vincent, P., Larochelle, H., Bengio, Y., and Manzagol, P.-A. (2008). Extracting and composing robust features with denoising autoencoders. *Proceedings of the 25th International Conference on Machine Learning*, pages 1096–1103.
- [Xu et al., 2024] Xu, D. et al. (2024). FITS: Modeling time series with $o(n)$ complexity. *arXiv preprint arXiv:2307.03750*.
- [Zhang et al., 2017] Zhang, D., Sun, Y., Eriksson, B., and Balzano, L. (2017). Deep unsupervised clustering using mixture of autoencoders. *arXiv preprint arXiv:1712.07788*.
- [Zhang et al., 2016] Zhang, Y., Zhou, D., Chen, S., Shen, J., and Myte, R. (2016). Single-image crowd counting via multi-column convolutional neural network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 589–597.
- [Øyvind Rønningstad and Nordic Semiconductor, 2021] Øyvind Rønningstad and Nordic Semiconductor (2021). zcbor: A low-footprint cbor library and code generator for c. <https://github.com/NordicSemiconductor/zcbor>.